# Project Plan Document

IndustryApp – post-accident repair offer platform · PA1

**Authors:** Ömer Efe Öztürk, Bahadır İhsan Herdem · **Course:** BIL 482 Design Patterns · **Version:** 1.0 · **Date:** 05/10/2026

## Table of Contents

1. Document Task Matrix
2. Project Scope
3. Project Organization – People
4. Objectives
5. Key Phases and Timeline
6. Resource Planning
7. Risk Management
8. Communication Plan
9. Change Management Plan
10. Budget Plan
11. Acceptance Tests and Acceptance Criteria

## 1. Document Task Matrix

| # | Task in creating this document | Responsible | Reviewer |
|---|---|---|---|
| T1 | Project scope and objectives | Bahadır İhsan Herdem | Ömer Efe Öztürk |
| T2 | Roles and responsibilities | Ömer Efe Öztürk | Bahadır İhsan Herdem |
| T3 | Phases, timeline and deliverable mapping | Bahadır İhsan Herdem | Ömer Efe Öztürk |
| T4 | Resource planning | Ömer Efe Öztürk | Bahadır İhsan Herdem |
| T5 | Risk management | Ömer Efe Öztürk | Bahadır İhsan Herdem |

| # | Task in creating this document | Responsible | Reviewer |
|---|---|---|---|
| T6 | Communication and change management plans | Bahadır İhsan Herdem | Ömer Efe Öztürk |
| T7 | Budget (effort) plan | Ömer Efe Öztürk | Bahadır İhsan Herdem |
| T8 | Acceptance tests and criteria | Bahadır İhsan Herdem | Ömer Efe Öztürk |
| T9 | Final review, formatting, GitHub upload | Ömer Efe Öztürk | Bahadır İhsan Herdem |

## 2. Project Scope

The project delivers a production-ready web platform for post-accident repair offers, covering four core use cases and one supporting admin flow.

**In scope:** UC-01 role-based registration and login, UC-02 accident report creation, UC-03 job notification and offer submission, UC-04 offer selection and contact reveal, YA-01 mechanic approval and catalog management; in-app and e-mail notifications; security hardening; production deployment on one server; PA1–PA3 documents and a test report.

**Out of scope:** payment and commission, ratings and reviews, in-app messaging, SMS provider, native mobile app, insurance integration, map-based search, multi-server scaling. These are listed as Future Work.

## 3. Project Organization – People

The team has two members; each member owns a set of areas and reviews the other's work.

- **Ömer Efe Öztürk — Backend Lead and DevOps.** Domain model, Application services, Infrastructure (EF Core, Identity, events, notifications), database scripts, Docker images, server setup, backups. Coordinates phase planning and tags releases.
- **Bahadır İhsan Herdem — Design Owner, Frontend Lead and QA Lead.** Keeps the Software Design Document as the single source of truth, owns the PA documents and UML diagrams, builds the React frontend, writes the test plan and the PA3 test report.
- **Both members:** every pull request is reviewed and merged by the member who did not write it; both implement and test pattern classes in their areas; both run the final demo.

- **Instructor (stakeholder):** sets the assignments, receives the PA packages through GitHub and evaluates the deliverables and the demo.

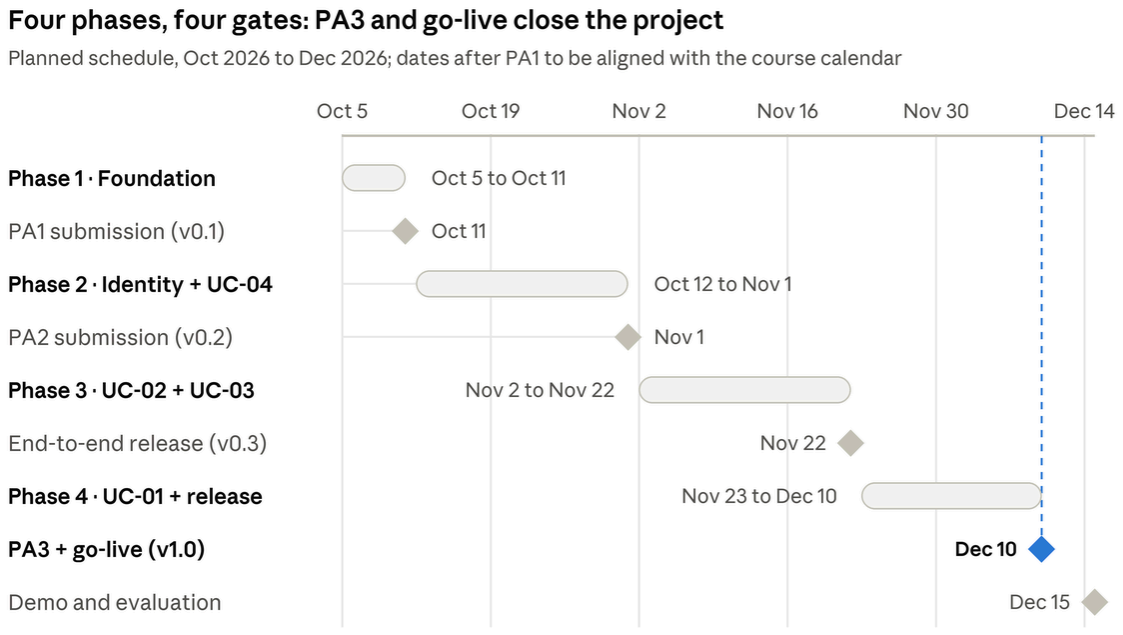
## 4. Objectives

Objectives for project execution and management:

1. Close each of the four phases with a working, tagged release ( `v0.1` , `v0.2` , `v0.3` , `v1.0` ) and submit each PA on its date.
2. Apply one definition of done to every step: `dotnet build` passes with warnings as errors, `npm run build` passes, related unit tests are green, every pattern class carries its pattern comment, no value is hard-coded (settings come from `appsettings.json` ), and the commit carries its UC number.
3. Review every pull request by the other member before it reaches `main` .
4. Keep documentation ahead of code: a change is written into the Software Design Document first, then implemented.
5. Keep the workload balanced and visible through the document task matrices, `TASK_MATRIX.md` and per-member commit history.

## 5. Key Phases and Timeline

The project runs in four phases from 05/10/2026 to 10/12/2026; each phase ends with a tagged release and a delivery gate. Dates after PA1 are planned and will be aligned with the course calendar.




IndustryApp schedule · 4 phases, 4 delivery gates

Requirements and design happen in Phase 1 (PA1), development in Phases 2–3, testing continuously and in full in Phase 4, deployment in Phase 4.

1. **Phase 1 – Foundation (05/10/2026 – 11/10/2026):** repository and 6-project solution, the full domain model with pattern skeletons, the database schema with reference data, the event infrastructure, an empty API with Swagger and `/health`, the React skeleton. The PA1 documents are written in parallel. *Gate:* `v0.1` and PA1 submission.
2. **Phase 2 – Identity and UC-04 (12/10/2026 – 01/11/2026):** JWT login, registration with Factory Method and legal consent, offer sorting (Strategy), offer selection (State + Command) with undo, contact reveal (Proxy), real-time notifications (Observer). *Gate:* `v0.2` and PA2 submission.
3. **Phase 3 – UC-02 and UC-03 (02/11/2026 – 22/11/2026):** damage catalog (Composite), photo pipeline (Decorator), file storage (Adapter), report creation (Builder), matching (Strategy + Chain of Responsibility), notification channels (Adapter + Circuit Breaker), offer submission with price breakdown (Decorator), mechanic approval. *Gate:* `v0.3`, the whole flow works with real data, the local test script is deleted.
4. **Phase 4 – UC-01 completion, administration and release (23/11/2026 – 10/12/2026):** role dashboards (Abstract Factory), profile and category management, security hardening, all automated tests and the test report, Docker images, server deployment, backups, PA3 documents. *Gate:* `v1.0`, live system and PA3 submission.

**Deliverables mapped to the timeline:**

- **PA1 package** (5 documents) → end of Phase 1 → 11/10/2026 (the PA1 submission is due by the end of that day)
- **PA2 package** (SDD for UC-04, source code `v0.2`, README, TASK_MATRIX) → end of Phase 2 → 01/11/2026
- **End-to-end release** `v0.3` (internal milestone) → end of Phase 3 → 22/11/2026
- **PA3 package and live system** (updated SDD, test report, source code `v1.0`) → end of Phase 4 → 10/12/2026
- **Project closure date:** 10/12/2026, when `v1.0` is tagged and PA3 is submitted.
- **Project evaluation date:** 15/12/2026 — demo in production and the team's retrospective against the success criteria.

## 6. Resource Planning

All development tools and libraries are free and open source; only the production server, the domain and an SMTP account have a running cost, and they are needed in Phase 4.

- **People:** two developers (see section 3).

- **Backend:** .NET 8 SDK, ASP.NET Core Web API, Entity Framework Core 8 + Npgsql, ASP.NET Core Identity + JWT, SignalR, FluentValidation, Polly (resilience), SixLabors.ImageSharp, MailKit, Serilog, Swashbuckle. Accessed through NuGet.
- **Frontend:** Node 20, React 18 + TypeScript + Vite, React Router 6, TanStack Query 5, Axios, React Hook Form + Zod, MUI 5, @microsoft/signalr. Accessed through npm.
- **Database:** PostgreSQL 16, run as a Docker container locally and in production.
- **Testing:** xUnit, NSubstitute, FluentAssertions, Testcontainers (real PostgreSQL in tests); Mailpit as a local test SMTP server.
- **Collaboration:** Git and GitHub (repository, branches, pull requests, issues), each member with an individual account.
- **Deployment:** Docker, Docker Compose, Caddy (reverse proxy with automatic HTTPS); one Ubuntu LTS server with 2 vCPU and 4 GB RAM.
- **External services (decision pending):** domain name and server provider, SMTP provider. Chosen by the start of Phase 4; the SMTP account is needed for e-mail tests in Phase 3 (Mailpit covers development until then).

## 7. Risk Management

- **R1 – Events lost on restart.** The event queue is in memory, so unprocessed events vanish if the process stops. *Likelihood:* low · *Impact:* medium. *Mitigation:* accepted for the single-server first release; the REST API stays the source of truth and clients refresh on reconnect; a Transactional Outbox is planned as Future Work.
- **R2 – E-mail provider fails or is not chosen in time.** *Likelihood:* medium · *Impact:* medium. *Mitigation:* Circuit Breaker + Retry keep in-app notifications flowing; Mailpit for development; provider decided before Phase 3 ends.
- **R3 – Team workload imbalance or a member becomes unavailable** (exams, illness, leaving). *Likelihood:* medium · *Impact:* high. *Mitigation:* task matrices make the split visible; cross-review of every pull request spreads knowledge; the design document lets one member continue the other's area.
- **R4 – Deployment problems** (DNS, certificates, server, migrations). *Likelihood:* medium · *Impact:* high. *Mitigation:* deploy early in Phase 4, health checks, migrations bundle instead of migrate-on-start, a tested backup restore, full rehearsal the day before the demo.
- **R5 – Unfamiliar technology** (SignalR, Polly, Testcontainers). *Likelihood:* medium · *Impact:* medium. *Mitigation:* each new technology starts from a working skeleton and is tried in a small spike at the start of its phase.

- **R6 – Schedule slip around exam weeks.** *Likelihood:* high · *Impact:* medium. *Mitigation:* every phase ends with a working tag; UC-04 is built first for PA2; nice-to-have items move to Future Work instead of delaying a gate.
- **R7 – Concurrency errors** (two selections at once). *Likelihood:* low · *Impact:* high. *Mitigation:* partial unique index plus optimistic concurrency ( `xmin` ), covered by an integration test (planned test TC-04.2.3).
- **R8 – Personal data leak** (phone numbers, photo GPS). *Likelihood:* low · *Impact:* high. *Mitigation:* contact details only through the protection proxy, EXIF stripping, phone masking in logs, KVKK consent records.
- **R9 – Changed requirements from the instructor.** *Likelihood:* medium · *Impact:* medium. *Mitigation:* handled through the change management plan (section 9).

## 8. Communication Plan

**Within the team:**

- **Weekly meeting** (about 1 hour, in person or online): progress against the phase checklist, blockers, the next week's tasks.
- **Daily asynchronous updates** in the team's messaging group: what was done, what is next, blockers.
- **GitHub as the record of work:** one issue per task with its UC number, a branch per issue ( `uc-0n/<short-name>` ), a pull request reviewed by the other member, UC-numbered commits.
- **Phase review** at the end of each phase: walk through the phase checklist together, merge to `main` , tag the release.

**With stakeholders:**

- The instructor receives each PA package through the GitHub repository; if it is private, the instructor is given access.
- Questions to the instructor go through the course's official channel (class hours or e-mail), asked by either member and shared with the other.
- The final demo is held in the production environment.

## 9. Change Management Plan

The Software Design Document is the single source of truth, so every change goes through it before it reaches code.

1. **Request:** the change (new requirement, instructor feedback, technical discovery) is opened as a GitHub issue labelled `change` , naming the affected UC, FR or PAT codes.

2. **Impact analysis:** the requester notes the affected documents, classes, tests and phase dates, and whether a PA deliverable is touched.
3. **Decision:** both members agree in the weekly meeting or in the issue. Changes that would move a PA date or drop a feature are confirmed with the instructor first.
4. **Update documents first:** the Software Design Document (and the SRS if requirements change) is updated, keeping the UC / FR / PAT numbering.
5. **Implement and test:** the change is coded on its own branch with a UC-numbered commit and reviewed by the other member.
6. **Record:** the change is added to the change log that goes into the PA3 design document.

Settings that are configurable by design (undo window, expiry time, matching strategy, limits) are changed only in `appsettings.json` and do not need this process.

## 10. Budget Plan

The main budget is effort: 2 members × about 10 hours per week × 9.5 weeks = about **190 person-hours**, planned as below. Money is needed only for production infrastructure.

**Effort by phase and member:**

- **Phase 1 – Foundation (1 week, 20 h, 10%):** Efe 10 h (repository, solution, domain model, database, event infrastructure) · Bahadır 10 h (PA1 documents, frontend skeleton, domain unit tests).
- **Phase 2 – Identity and UC-04 (3 weeks, 60 h, 32%):** Efe 30 h (identity, JWT, Factory Method registration, selection command, contact proxy, SignalR) · Bahadır 30 h (auth and UC-04 screens, tests, PA2 design document).
- **Phase 3 – UC-02 and UC-03 (3 weeks, 60 h, 32%):** Efe 32 h (photo pipeline, storage, matching, notification channels, Circuit Breaker) · Bahadır 28 h (report wizard, mechanic and admin screens, tests).
- **Phase 4 – UC-01 completion and release (2.5 weeks, 50 h, 26%):** Efe 27 h (Abstract Factory, security hardening, Docker, server, backups) · Bahadır 23 h (profile and admin screens, test report, PA3 documents, demo script).

**Effort by activity:** requirements and design documents 29 h (15%) · backend development 66 h (35%) · frontend development 47 h (25%) · testing and test report 29 h (15%) · deployment and operations 10 h (5%) · meetings and project management 9 h (5%).

**Monetary resources:** all development tools, libraries and GitHub are free. Three items cost money in Phase 4: the production server (2 vCPU, 4 GB RAM), the domain name, and the SMTP sending service. Their amounts depend on the providers, which are still open

decisions; the team fixes the amounts and who pays when the providers are chosen, before Phase 4 starts.

About 10% of each phase's hours are kept as buffer for exam weeks and rework.

## 11. Acceptance Tests and Acceptance Criteria

The system is accepted when the eight end-to-end scenarios below pass in the production environment and the acceptance criteria hold. Results are recorded in `docs/PA3/TestReport.xlsx` (test no, name, UC/FR, steps, expected, actual, tester, date dd/MM/yyyy, environment, log/traceId, status Pass/Fail/Blocked, note).

**Acceptance tests (end-to-end, run manually in production):**

1. **E2E-01 (UC-01, YA-01):** a mechanic registers, the admin approves, the mechanic logs in again → an approval e-mail arrives and the mechanic sees the job list.
2. **E2E-02 (UC-02, UC-03):** a customer creates a report with 3 photos → the report is Open and matched mechanics get a pop-up within 3 seconds.
3. **E2E-03 (UC-03):** two mechanics submit offers → the offers appear on the customer's screen without a page refresh; the mechanics cannot see each other's offers.
4. **E2E-04 (UC-04):** the customer selects an offer → the winner is notified "selected", the other "not selected"; phone numbers appear on the match screen.
5. **E2E-05 (UC-04):** the customer undoes the selection within 10 minutes → the report is Open again, offers are Pending, mechanics are notified.
6. **E2E-06 (UC-03):** the SMTP setting is deliberately broken and a report is created → in-app notifications still arrive; `CircuitOpen` records appear in `notification_deliveries`.
7. **E2E-07 (UC-04):** an unmatched mechanic calls `/api/reports/{id}/contact` → 403 or a masked number.
8. **E2E-08 (UC-02):** a report waits without a selection for the expiry time (shortened for the test) → the report becomes Expired and the customer is notified.

**Acceptance criteria:**

1. All 8 end-to-end scenarios pass in production.
2. All 15 automated tests (unit and integration) are green; coverage ≥ 70% for Domain and Application, ≥ 90% for pattern classes.
3. Report creation responds in under 500 ms (p95), independent of matching and notification.
4. No API response gives an unmatched user the other party's real phone number.

5. Every item of the security and release checklist is ticked: HTTPS + HSTS, rate limiting, CORS, log masking, secrets only in environment variables, nightly backup restored once.
6. Every phase gate is met: `v0.1` foundation, `v0.2` + PA2, `v0.3` end to end, `v1.0` + PA3 + live system.